

Python Programming with Artificial Intelligence

Introduction

Huangxun Chen @IoT&AI Thrust



Today's Outline

- Course Information
- Introduction to Python
- Python and IDE Setup
- GitHub Copilot Setup
- Simple LLM API Call demo



Course Information

Class Schedule: 15 June -- 27 July

- Time: Mon/Wed, 09:00AM - 11:50AM (10-min break for every 50min)
- Location: Rm 202, W4

Instructor: Huangxun Chen

TA:

- Jialuo He, PhD, Email: jhe212@connect.hkust-gz.edu.cn
- Zhixun Chen, PhD, Email: zchen233@connect.hkust-gz.edu.cn



Huangxun Chen, HKUST(GZ)

B.S. @SJTU Ph.D.@HKUST Researcher@Huawei HK Assistant Professor@HKUST(GZ)



2015



2020



2020-2023



2023-now

Research Interest

- Efficient Edge/Embedded AI
高效边缘/嵌入式智能
- Agentic AI (for domain-specific workflows and everyday scenarios)
智能体系统 (面向领域特定工作流和日常场景)
- Security&Privacy in Sensing/Perception Systems
感知系统中的安全与隐私

[Homepage] www.chenhuangxun.com

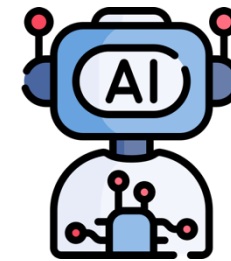
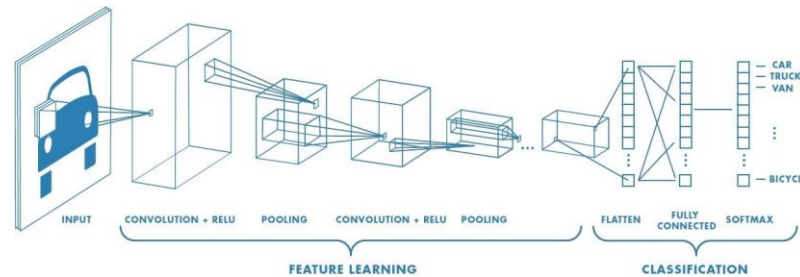
[Email] huangxunchen@hkust-gz.edu.cn [Office] E1-606

- ACM SIGCOMM 2022 Best Paper Award (First author)
- ACM IMWUT 2025 Distinguished Paper Award
- Huawei Innovation Pioneer President Award
- ACM IMWUT, Associate Editor
- ACM SIGBED China, Executive Committee Member
- CCF TCPC, Executive Committee Member
- National-Level Overseas Young Talent
- Guangdong Pearl-River Top Young Talent
- Invited talk at MobiArch 2025, IETF 115
- MobiSys/IWQoS/ICDCS/AAAI/etc, TPC Member

Course Goals

By the end of this course, students will be able to:

- Develop a strong foundation in advanced Python programming concepts
- Learn to implement core Machine Learning and Deep Learning models using Python
- Gain proficiency in building Python-based AI applications by integrating models, data pipelines, APIs, and evaluation workflows
- Acquire hands-on experience with modern AI-assisted software development tools and learn how to use them effectively for project development



Course Overview and Agenda

Week	Date	Title	
1	2026-06-15	Introduction	
1	2026-06-17	Advanced Python Fundamentals-I	Part 1: Advanced Python Fundamentals
2	2026-06-22	Advanced Python Fundamentals-II	
2	2026-06-24	Advanced Python Fundamentals-III	
3	2026-06-29	Machine Learning I – workflow, supervised v.s. unsupervised	Part 2: Python for ML/DL/LLM
3	2026-07-01	Machine Learning II - evaluation, regularization, ensembles, ...	
4	2026-07-06	Deep Learning I: PyTorch (tensors, autograd, NN)	
4	2026-07-08	Deep Learning-II: architectures, optimization, training, debugging	
5	2026-07-13	Deep Learning-III: LLMs, transformers, pretrain, fine-tuning, inference	
5	2026-07-15	AI-powered Application I: prompting, structured outputs, ...	Part 3: AI-powered and Agentic Applications
6	2026-07-20	AI-powered Applications II: RAG, tool use, agentic workflows, etc, ...	
6	2026-07-22	AI-powered Applications III: AI-assisted Coding	
7	2026-07-27	Final Project Presentation - presentations, reflection, wrap-up	Part 4: Assessment
8	TBD	Final Exam	

Grading Scheme

Course Component	Weight (%)
Attendance	10%
In-Class Practices	20%
Assignment (2-3 times)	20%
Final Exam	30%
Final Project (Report+Code+Presentation)	20%

Attendance:

- Attendance will be recorded **every** class
- Everyone has **2 free absences**
- No attendance or in-class practice in this first class

In-class Practices

- Each class will include some short practice exercises
- 1-2 practice per class will count toward to your grade
- Make sure to complete and submit these graded practices

This is a group project. Each team should consist of **1–3 students**.

- **Design and build** an interesting and creative system
- **Write a report** that explains your design and findings
- **Present in class:** share your system design, give a demo, and highlight your contributions
- **Submit your code** for evaluation and reproducibility

DDL: (Week 13) July 27, 2026

A suggested list of final project topics will be provided in the coming weeks.

Self-proposed topics are welcome as long as they are aligned with the course scope.

The final exam is a closed-book assessment

Goal: Evaluate students' individual mastery of the core concepts covered in the course.

Scope:

- Python programming fundamentals
- Machine learning and deep learning workflows
- LLM-related concepts and practices, etc.

The exam will NOT require memorization of specific library APIs.

Week 8. Exact date/time TBD by a majority vote among enrolled students, subject to classroom and administrative arrangements.

Overall Assessment

Cumulative Score (out of 100)	Possible Grades
90~100	A-/A/A+
70~90	B-/B/B+/A-
50~70	C-/C/C+/B-
40~50	D/C-
<40	F

Note:

1. The assessment primarily focuses on **how well the expected learning outcomes are achieved**.
2. The **mapping** provided above is from past experiences and should be considered as **a preliminary guide**.

Important Notes

- Do not plagiarize. Academic honesty is essential. Any copied work will result in serious consequences.
- Teaching Style: ~60% Lectures w/ Concepts/Examples + ~40% Practices

- We are open-minded to Generative AI tools.
- This is a fundamental programming course, and one of the important purpose is for you to develop your own coding skills
- Understanding the underlying principles helps you communicate better with AI and enables more effective use of AI tools.
- Remember: AI should assist, not replace your thinking

“What I cannot create, I do not understand.”
---Richard Feynman



More Questions?



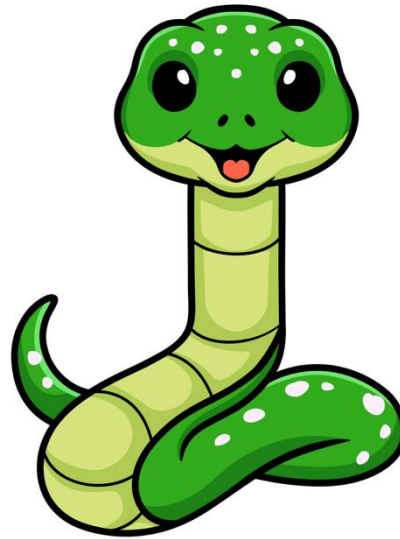
Introduction to Python

One of the most popular programming language

May 2026	May 2025	Change	Programming Language		Ratings	Change
1	1		 Python		19.98%	-5.37%
2	3	▲	 C		11.55%	+1.84%
3	4	▲	 Java		7.94%	-1.37%
4	2	▼	 C++		7.92%	-2.02%
5	5		 C#		5.41%	+1.19%
6	6		 JavaScript		3.08%	-0.60%
7	8	▲	 Visual Basic		2.90%	+0.28%
8	12	▲▲	 R		1.77%	+0.31%
9	10	▲	 SQL		1.57%	-0.33%
10	9	▼	 Delphi/Object Pascal		1.44%	-0.85%
11	11		 Fortran		1.22%	-0.55%
12	14	▲	 Scratch		1.18%	-0.16%
13	16	▲	 Perl		1.18%	-0.02%
14	15	▲	 PHP		1.15%	-0.07%
15	19	▲▲	 Rust		1.14%	+0.21%
16	7	▼▼	 Go		1.12%	-1.58%
17	18	▲	 Assembly language		1.02%	+0.05%
18	23	▲▲	 Swift		0.93%	+0.16%
19	13	▼▼	 Ada		0.92%	-0.50%
20	17	▼	 MATLAB		0.89%	-0.13%

One of the most popular programming language

- Invented in the Netherlands by Guido van Rossum in 1989, released in 1991
- The name “Python” was inspired by the British comedy group Monty Python, not the snake



Beginner-friendly, entry-level language that is originally designed for education

- **High Readability**
 - Keywords that meaningful in English
 - New lines to finish a command and indentation to define the scope
- **Fast development and experimentation**
 - Vast Libraries Support: over 600,000 packages in PyPI
 - High-level built-in functionalities (implemented in C)
 - Script language with interactive mode for fast testing and debugging
- **Cross-platforms compatibility:** works seamlessly on macOS, Linux, windows, and even Raspberry Pi
- **Large and active community:** abundant tutorials, resources, and support

■ Programming Language

- **Program:** A program is simply a sequence of instructions written in a programming language that tells the computer what to do step by step.
- **Instruction:** The smallest operation a computer can execute, typically consisting of an Opcode (what action to perform) and an Operand (the data or location on which the action is applied).

Example:

```
1  mov 3, reg1      # Move the number 3 into register 1
2  mov 5, reg2      # Move the number 5 into register 2
3  add reg1, reg2    # Add the values in reg1 and reg2
```

Programming Language: A formal system of notation used to write computer programs

- **Machine Language:** Instructions written directly in binary (0s and 1s), executed by the hardware
- **Assembly Language:** A symbolic representation of machine language, where opcodes and operands are written with human-readable symbols (e.g., MOV, ADD)
- **High-Level Programming Language:** Provides strong abstraction from hardware details. Easier for humans to read and write, but must be **compiled or interpreted** into machine code for execution

Questions: Why High-level?

High-level Language

Low-level Language



Python/C++/Java/C

Assembly Language

Machine Language

$X = 2 \times (3 + 4)$

```
# Store the result of 3+4 in register 0
add r0, 3, 4;
# Store the result of 2xr0 in register 0
mul r0, r0, 2;
return r0
```

```
00000000 00100 00011 000
00000 0110011
```

```
0000001 00010 00000 000
00000 0110011
```

Programming Language

Print all prime numbers between 0 and 100

```
# Loop through numbers from 2 to 99 (inclusive)
for i in range(2, 100):
    # Loop through numbers from 2 to i-1 (inclusive)
    for j in range(2, i):
        # If i is divisible by j, it is not a prime number, break the inner loop
        if i % j == 0:
            break
    # If the inner loop was not broken, i is a prime number
    else:
        print(i)
```

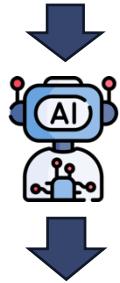
Python Code, only 6 lines

```
11 segment .text
12     global asm_main
13 asm_main:
14     enter    0,0          ; setup routine
15     pusha
16
17     mov     eax, Message
18     call    print_string
19
20     call    read_int       ; scanf("%u", & limit );
21     mov     [Limit], eax
22
23     mov     eax, 2         ; printf("2\n");
24     call    print_int
25     call    print_nl
26     mov     eax, 3         ; printf("3\n");
27     call    print_int
28     call    print_nl
29
30     mov     dword [Guess], 5 ; Guess = 5;
31
32 while_limit:
33     mov     eax, [Guess]
34     cmp     eax, [Limit]
35     jnbe    end_while_limit ; use jnbe since numbers are u
36
37     mov     ebx, 3         ; ebx is factor = 3;
38 while_factor:
39     mov     eax, ebx
40     mul     eax            ; edx:eax = eax*eax
41     jo      end_while_factor ; if answer won't fit in eax a
42     cmp     eax, [Guess]
43     jnb     end_while_factor ; if !(factor*factor < guess)
44     mov     eax, [Guess]
45     mov     edx, 0
46     div     ebx           ; edx = edx:eax % ebx
47     cmp     edx, 0
48     je      end_while_factor ; if !(guess % factor != 0)
49
50     add     ebx, 2         ; factor += 2;
51     jmp     while_factor
52 end_while_factor:
53     je      end_if        ; if !(guess % factor != 0)
```

Assembly Code, more than 100 lines

Programming Language

Print all prime numbers between 0 and 100



```
# Loop through numbers from 2 to 99 (inclusive)
for i in range(2, 100):
    # Loop through numbers from 2 to i-1 (inclusive)
    for j in range(2, i):
        # If i is divisible by j, it is not a prime number, break the inner loop
        if i % j == 0:
            break
    # If the inner loop was not broken, i is a prime number
    else:
        print(i)
```

Python Code, only 6 lines

📌 已置顶



Andrej Karpathy @karpathy · Jan 24, 2023

The hottest new programming language is English

💬 2289

🔄 8950

❤️ 7万

📊 1270万

🔖 ⬆

```
11 segment .text
12 global asm_main
13 asm_main:
14     enter    0,0          ; setup routine
15     pusha
16
17     mov     eax, Message
18     call    print_string
19
20     call    read_int       ; scanf("%u", & limit );
21     mov     [Limit], eax
22
23     mov     eax, 2         ; printf("2\n");
24     call    print_int
25     call    print_nl
26     mov     eax, 3         ; printf("3\n");
27     call    print_int
28     call    print_nl
29
30     mov     dword [Guess], 5 ; Guess = 5;
31
32 while_limit:
33     mov     eax,[Guess]
34     cmp     eax, [Limit]
35     jnbe    end_while_limit ; use jnbe since numbers are u
36
37     mov     ebx, 3         ; ebx is factor = 3;
38 while_factor:
39     mov     eax,ebx
40     mul     eax            ; edx:eax = eax*eax
41     jo      end_while_factor ; if answer won't fit in eax a
42     cmp     eax, [Guess]
43     jnb     end_while_factor ; if !(factor*factor < guess)
44     mov     eax,[Guess]
45     mov     edx,0
46     div     ebx            ; edx = edx:eax % ebx
47     cmp     edx, 0
48     je      end_while_factor ; if !(guess % factor != 0)
49
50     add     ebx,2          ; factor += 2;
51     jmp     while_factor
52 end_while_factor:
53     je      end_if        ; if !(guess % factor != 0)
```

Assembly Code, more than 100 lines

■ Programming Language

Programming Language: A formal system of notation used to write computer programs

- **Machine Language:** Instructions written directly in binary (0s and 1s), executed by the hardware
- **Assembly Language:** A symbolic representation of machine language, where opcodes and operands are written with human-readable symbols (e.g., MOV, ADD)
- **High-Level Programming Language:** Provides strong abstraction from hardware details. Easier for humans to read and write, but must be **compiled or interpreted** into machine code for execution

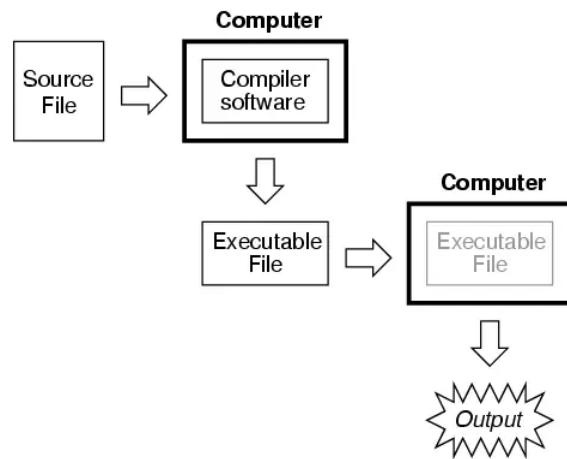
Questions: Why High-level?

You can focus on what to do, not on the low-level details of how the machine does it.

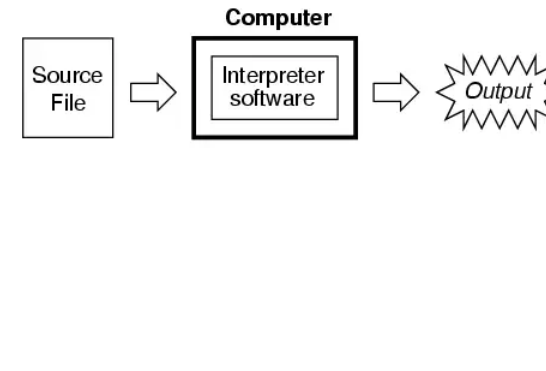
Interpreted vs. Compiled language

How high-level programming languages are translated into machine language

- **Compiled language:** The entire program is first translated into machine code, which the processor can run directly (e.g., C/C++, Go)
- **Interpreted language:** Code is executed line by line by an interpreter, without producing a separate machine code file (e.g., Java, Python, ..)



Compiled Languages

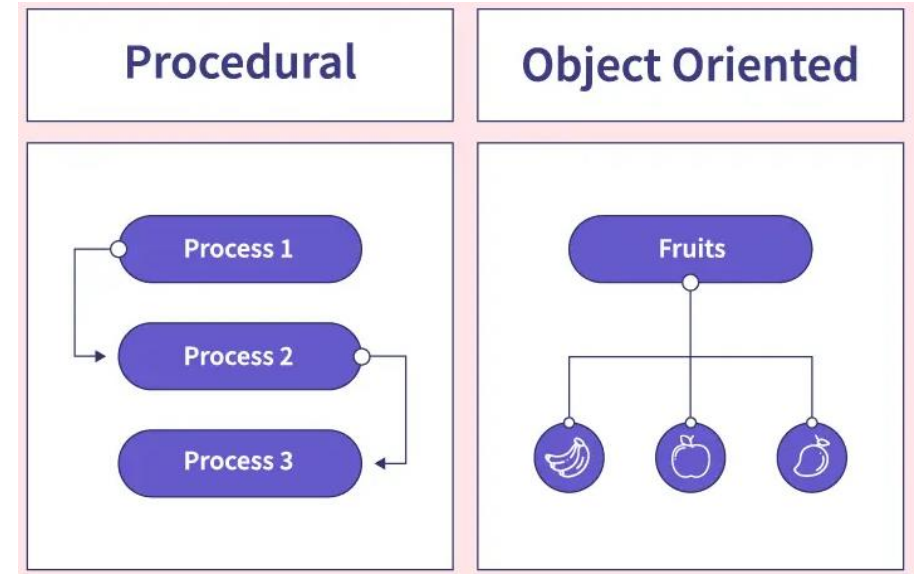


Interpreted Languages

Procedural vs Object-oriented language

Procedural programming languages

- Organizes code around procedures
- Follows a step-by-step, linear flow of control
- Limited support for code reuse and encapsulation
- Examples: C, Fortran



Object-oriented programming languages

- Organizes code around objects that bundle data and behavior
- Supports encapsulation, inheritance, and polymorphism
- Good for modeling real-world entities and interactions
- Examples: Java, C++, Python

- **Rich basic data types:** Integers, floats, strings, lists, dictionaries, etc
Dynamic typing: no need to declare variable types explicitly
- **Advanced features:** Generators for efficient iteration and list comprehensions for concise code
- **Clean error handling:** Built-in support for raising and catching exceptions
- **Object-oriented:** In Python, almost everything is treated as an object (even functions)
- **Automatic memory management:** No need to allocate and free memory manually

Sacrifices efficiency for simplicity

- **Slower execution:** Basic operations run less efficiently
- **Higher memory usage:** Not ideal when memory optimization is critical (e.g., mobile apps)

Solution: Implement efficiency-critical components in C/C++ and expose them through a Python interface

Python is suitable for ...

Anything not requiring efficiency for practical deployment

- **Research**
 - Data Process, Data Analysis, Calculation, ...
- **Automation Script**
 - OS, Database, ...
- **Prototype product development**
- **Machine Learning/Deep Learning**
- **Web Crawling**

$$\frac{\partial}{\partial a} \ln f_{a, \sigma^2}(\xi_1) = \frac{(\xi_1 - a)}{\sigma^2} f_{a, \sigma^2}(\xi_1) = \frac{1}{\sqrt{2\pi\sigma}} \exp\left(-\frac{(\xi_1 - a)^2}{2\sigma^2}\right)$$
$$\int_{\mathcal{R}_n} T(x) \cdot \frac{\partial}{\partial \theta} f(x, \theta) dx = M\left(T(\xi) \cdot \frac{\partial}{\partial \theta} \ln L(\xi, \theta)\right) = \int_{\mathcal{R}_n} T(x) \left(\frac{\partial}{\partial \theta} \ln L(x, \theta)\right) f(x, \theta) dx$$
$$\int_{\mathcal{R}_n} T(x) \cdot \left(\frac{\partial}{\partial \theta} \ln L(x, \theta)\right) \cdot f(x, \theta) dx = \int_{\mathcal{R}_n} T(x) \left(\frac{\partial}{\partial \theta} \ln L(x, \theta)\right) f(x, \theta) dx$$
$$\frac{\partial}{\partial \theta} \ln L(\xi) = \frac{\partial}{\partial \theta} \int_{\mathcal{R}_n} T(x) f(x, \theta) dx = \int_{\mathcal{R}_n} T(x) \frac{\partial}{\partial \theta} f(x, \theta) dx$$

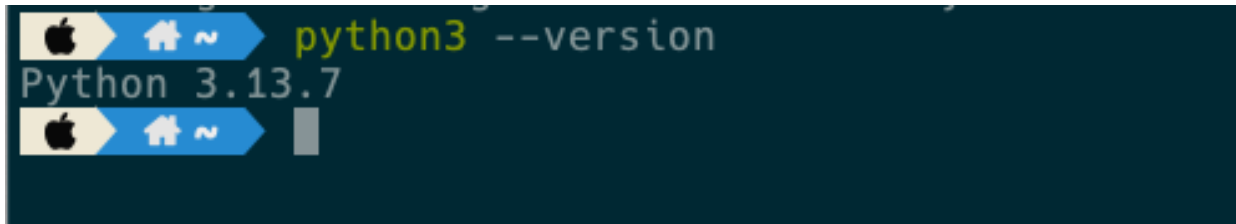




Python and IDE Setup

Python Setup

- Download and install Python
 - <https://www.python.org/downloads/>
- Check if the installation is successful



```
python3 --version
Python 3.13.7
```

- Two modes to run Python code: interactive & script

Start coding with Python – interactive mode

- Enter the interactive python interface

```
python3
Python 3.13.7 (v3.13.7:bcee1c32211, Aug 14 2025, 19:10:51) [Clang 16.0.0 (clang-1600.0.26.6)] on darwin
Type "help", "copyright", "credits" or "license" for more information.
>>> 
```

- Try printing something

```
>>> print("hello world")
hello world
>>> 
```

- Display the results instantly, suitable for simple, one-time computations

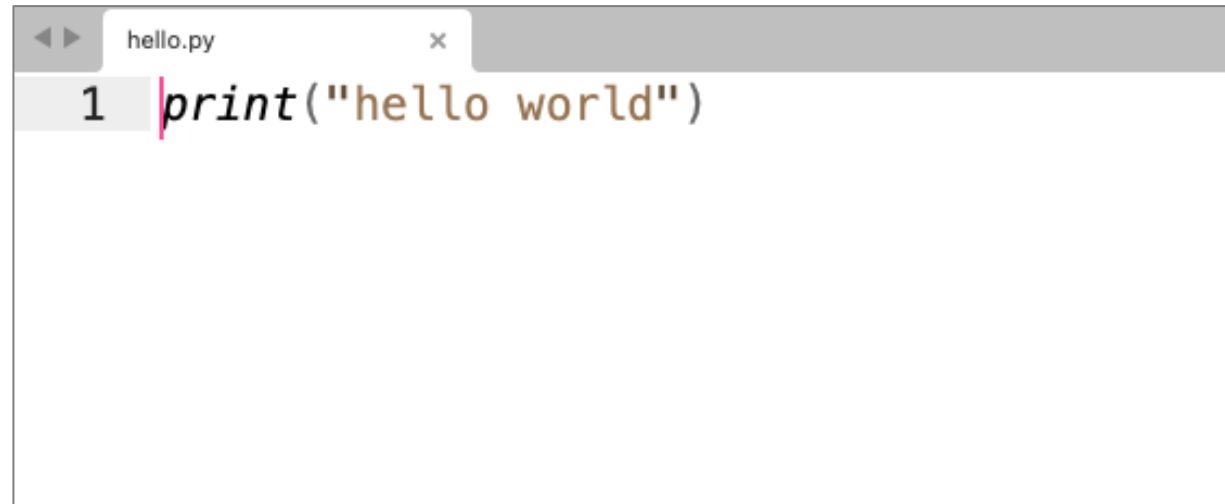
```
>>> 111*234
25974
```

- Exit the interactive interface

```
>>> exit()
```

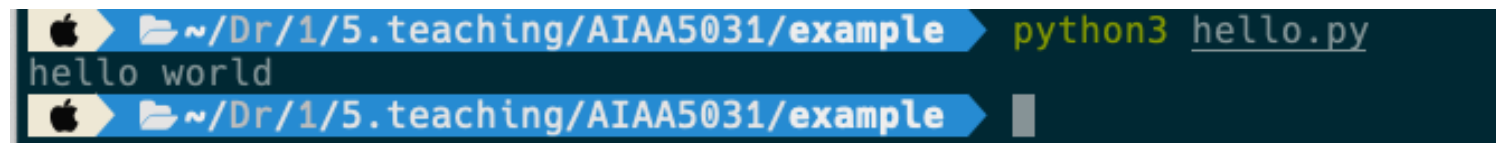
Start coding with Python – file mode

- Save the commands in a file and execute it
- Write the code in a .py file



```
hello.py x
1 print("hello world")
```

- Run the file to execute all commands at once



```
~/Dr/1/5.teaching/AIAA5031/example ➤ python3 hello.py
hello world
~/Dr/1/5.teaching/AIAA5031/example ➤
```

- **Integrated Development Environment (IDE):** A software application that combines key tools a developer needs to write, build, and debug software into a single, user-friendly interface.
- **Why using IDE?** It makes coding easier by offering:
 - Syntax highlighting: keywords and errors are clearly marked
 - Auto completion: helps you finish code faster
 - Debugging tools: step through your program to find mistakes
 - Project/file management: organized workspace for code
 - Environment management: easily manage libraries and dependencies
 - Remote server connection: run and test code on other machines

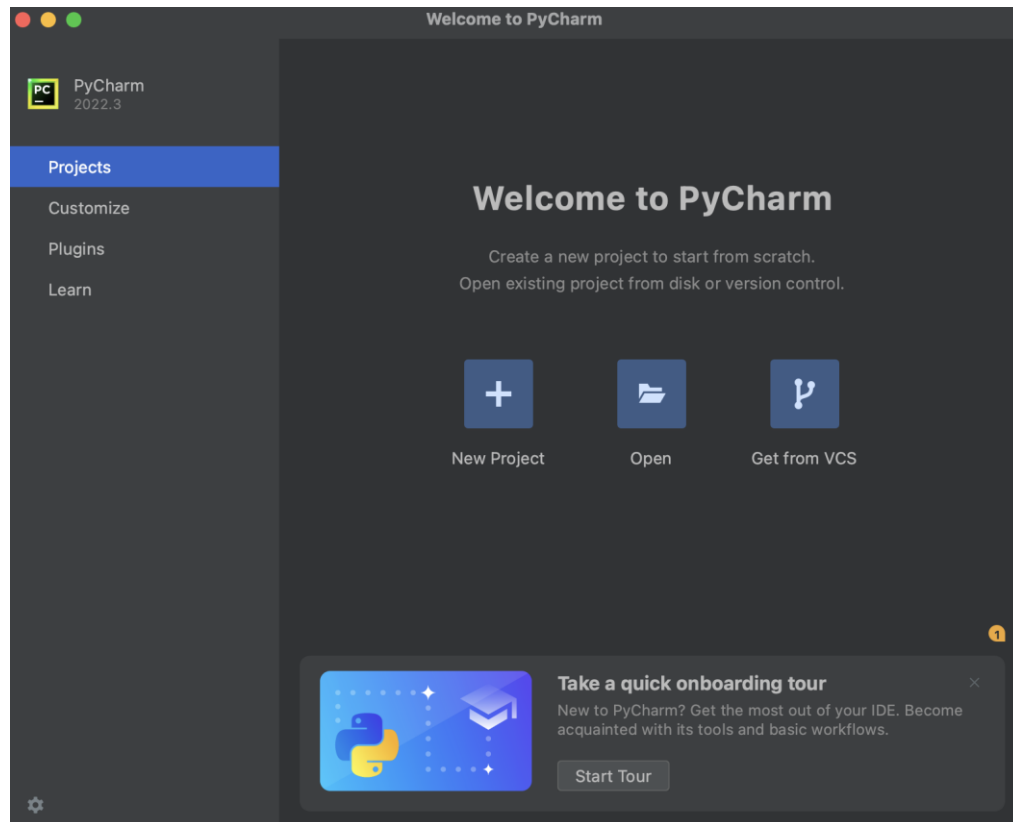
Recommended IDEs for Python

- PyCharm
- Visual Studio Code

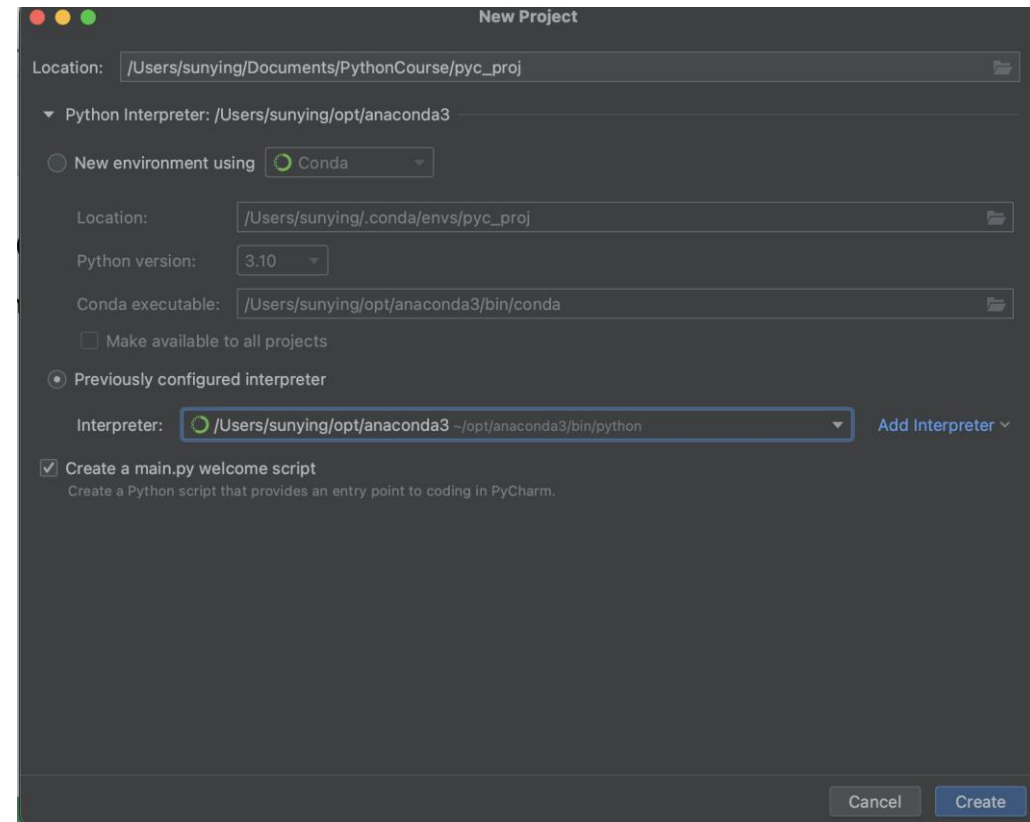
One of the most widely used IDEs for Python development

- All-in-one environment with integrated tools for Python
- Smart code editor with debugging and testing support
- Offers both open-source community version and commercial professional version
- **Beginner-friendly: convenient and easy to get started**
- **Download:** <https://www.jetbrains.com/pycharm/>
- **Education licenses:** <https://www.jetbrains.com/shop/eform/students>

Create a project



Choose path and interpreter



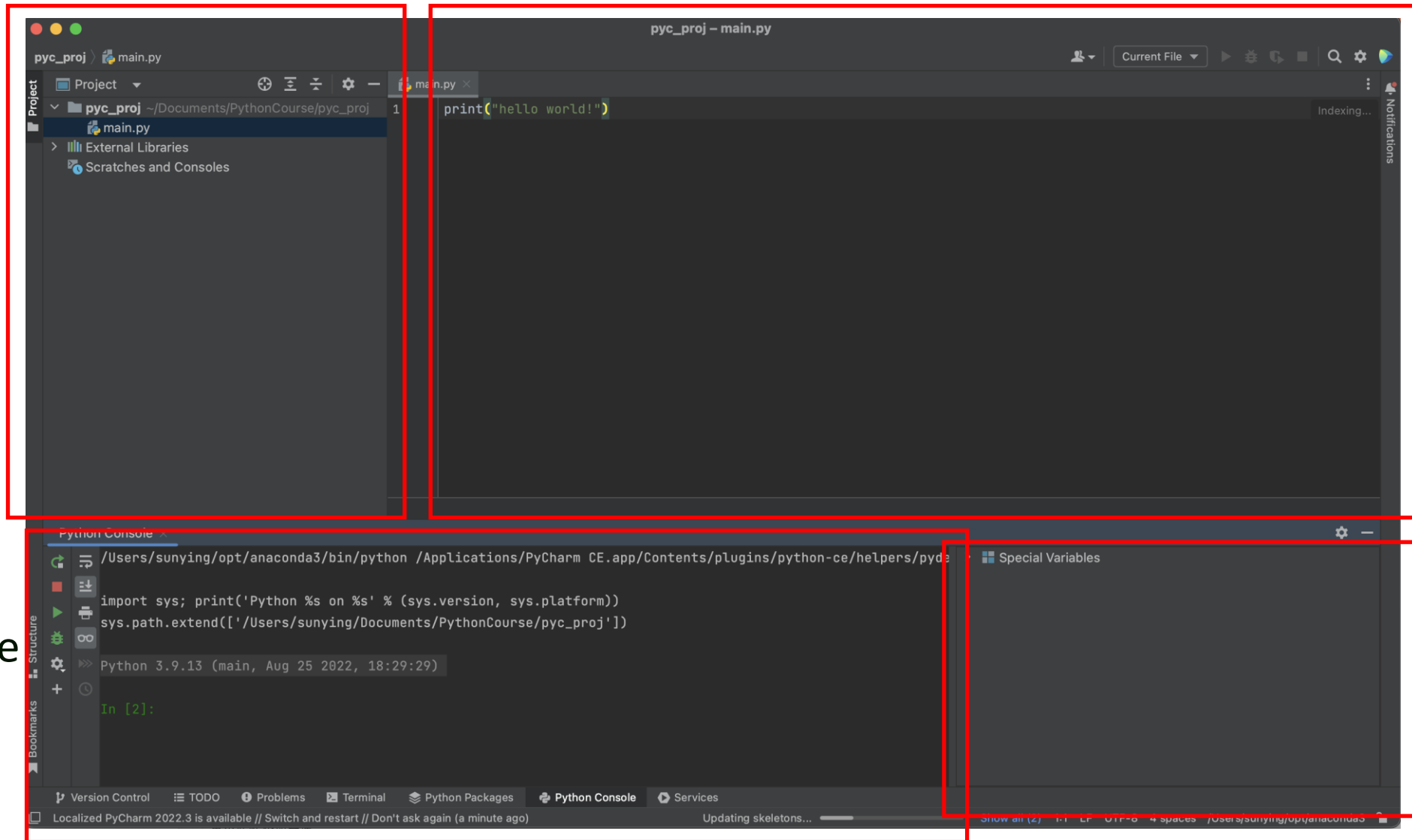
Can use conda environment

Editing area

Run

Debug

File tree



Python Console
Terminal

Variables

...

Tips for using PyCharm

- Change the keymaps of shortcuts
- Try connecting to a remote server (professional version only)

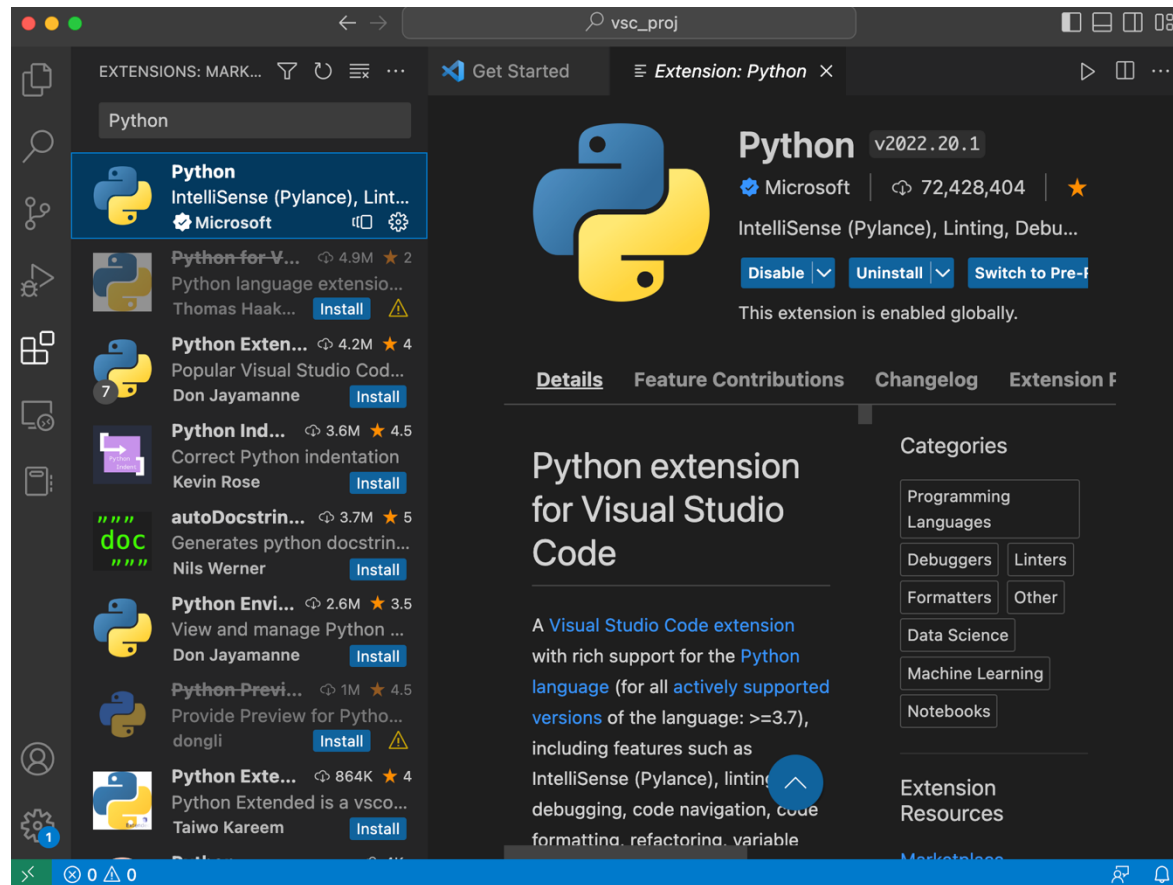
A lightweight yet powerful editor for Python and many other languages

- Highly customizable with extensions and themes
- Integrated terminal and Git support
- Debugging, linting, and testing tools available
- Free, open-source, and cross-platform

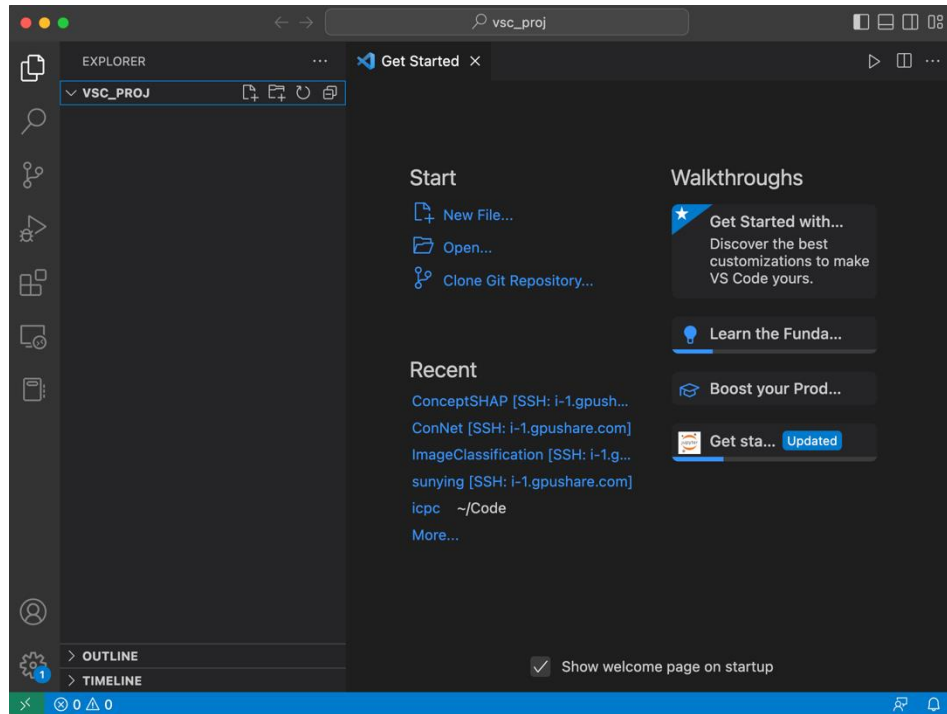
Popular among developers: versatile and widely used beyond Python

Download: <https://code.visualstudio.com/>

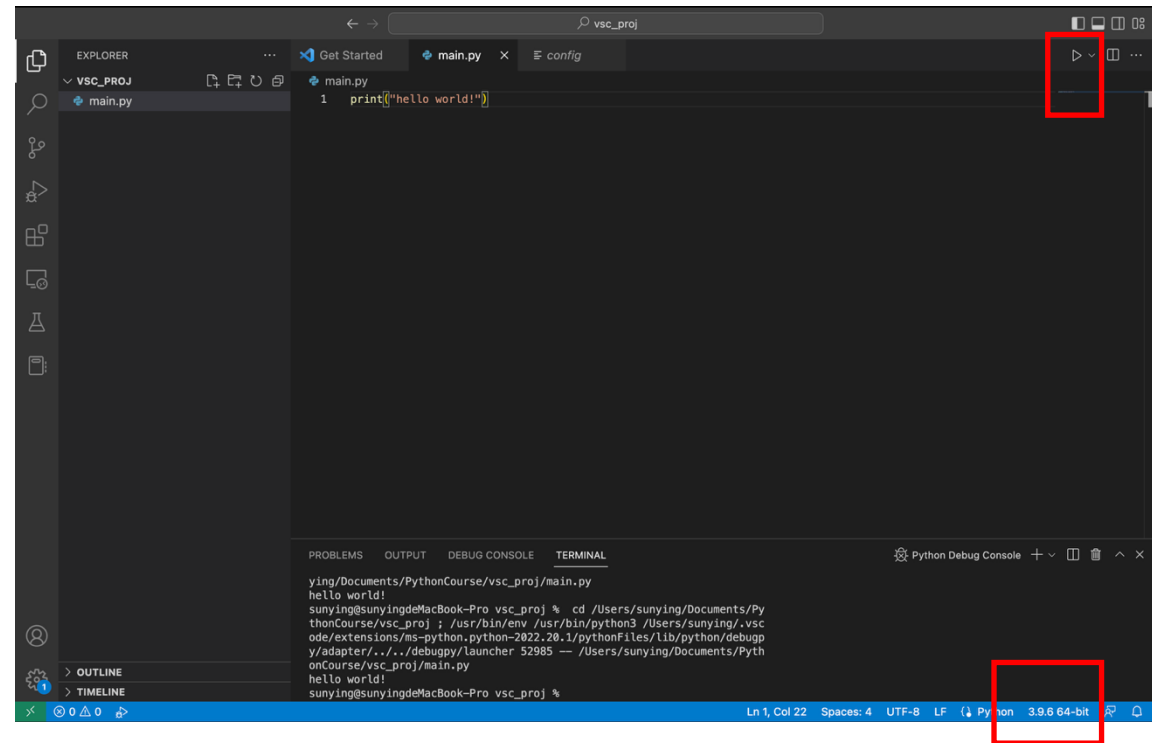
Install the Python extension



Open the path to work in



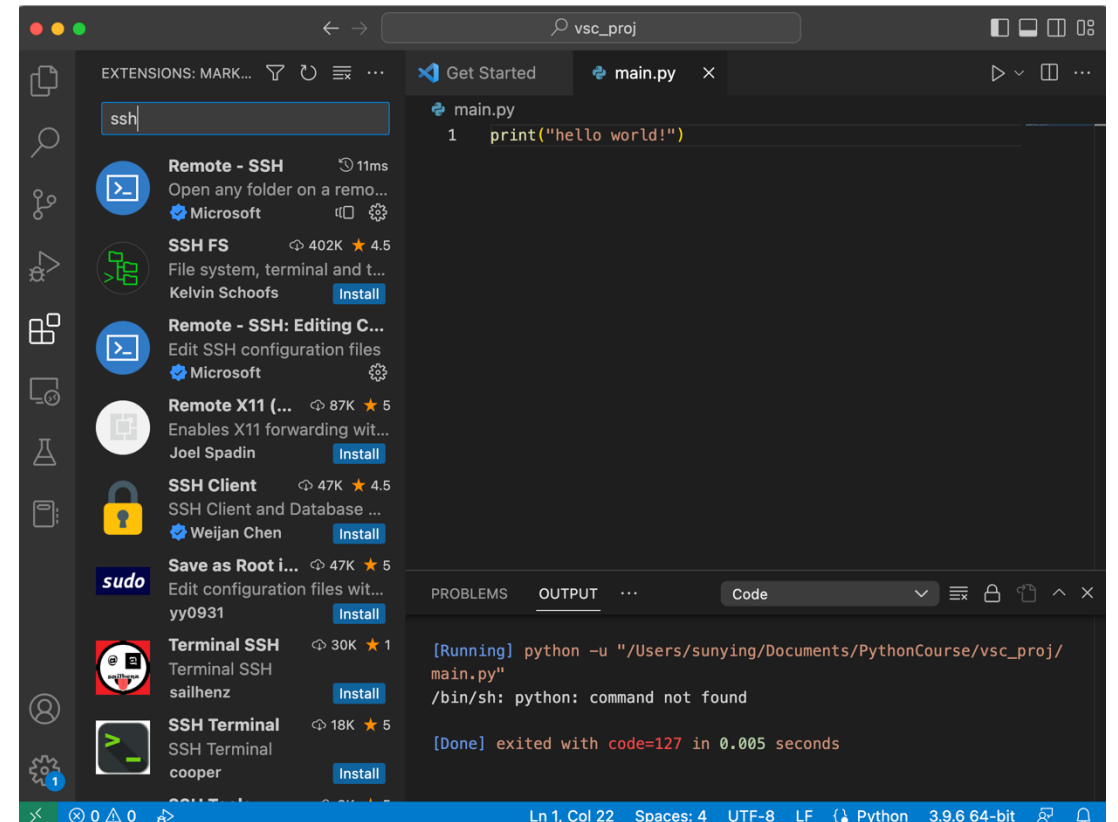
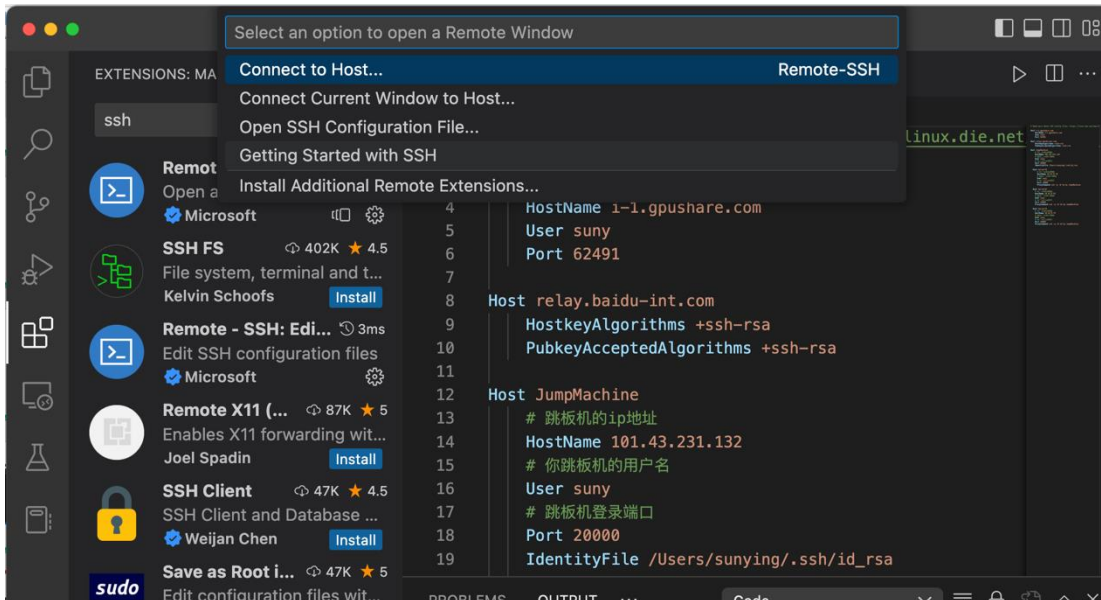
Create a file and Run



Select a different python environment

Connect to a remote server

- Install Remote SSH Extension
- Click SSH button
- Add Host in the configuration file
- Connect to the host



Example: Do some analysis for Fibonacci sequence

- Fibonacci sequence: 1, 1, 2, 3, 5, 8, ...
 - $a[n] = a[n-1] + a[n-2]$
- Analysis
 - `print a[1000000000]`

Interactive vs. Script Mode

Example: Do some analysis for Fibonacci sequence

- Fibonacci sequence: 1, 1, 2, 3, 5, 8, ...
 - $a[n] = a[n-1] + a[n-2]$
- Analysis
 - `print a[1000000000]`

```
4  lst = [0, 1]
5  for u in range(1, 1000000001):
6      lst.append((lst[u-1] + lst[u-2]) % 10000000000)
7
8  print(lst[1000000000])
```

Another 31s to execute the code

Interactive vs. Script Mode

Example: Do some analysis for Fibonacci sequence

- Fibonacci sequence: 1, 1, 2, 3, 5, 8, ...

➤ $a[n] = a[n-1] + a[n-2]$

- Analysis

➤ `print a[1000000000]`

➤ `Print a[999999999] + a[1002]`

```
4  lst = [0, 1]
5  for u in range(1, 1000000001):
6      lst.append((lst[u-1] + lst[u-2]) % 10000000000)
7
8  print(lst[1000000000])
9  print(lst[999999999] + lst[1002])
```

Another 31s to execute the code

Interactive mode can save time

```
Python 3.9.13 (main, Aug 25 2022, 18:29:29)
[Clang 12.0.0 ] :: Anaconda, Inc. on darwin
Type "help", "copyright", "credits" or "license" for more information.
>>> lst = [0, 1]
>>> for u in range(2, 100000001):
...     lst.append((lst[u-1] + lst[u-2]) % 10000000000)
...
>>> print(lst[1000000000])
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
IndexError: list index out of range
>>> print(lst[100000000])
7760546875
>>> print(lst[99999])
6278790626
>>> print(lst[99999] + lst[1002])
8451423002
>>> print(lst[9999999] * lst[10000])
7364937187632913750
>>>
```

31s for processing
Then any analysis

How to reuse the code
next time?

Interactive vs. Script Mode

Interactive	File
Instant feedback	No instant feedback
Run one line at a time	Execute the whole file
Not reusable	Reusable
Difficult to debug	Easy to debug

Observation:

- Interactive mode is quick and convenient for testing ideas, but not suitable for larger projects.
- Script mode is structured and reusable, but less flexible for immediate feedback

Motivation:

Can we combine the advantages of both?

This leads us to Jupyter Notebook.

Hybrid Approach: combine script and interactive modes by coding in blocks

Time consuming
data processing

Interactive analysis

Adding analysis without re-
executing the processing

```
jupyter_example.ipynb > print(lst[9])
+ 代码 + Markdown | ▶ 全部运行 | 清除所有单元格输出 | Outline ...
py37 (Python 3.7.13)
```

```
[1]: lst = []
    for i in range(100000000):
        lst.append(i)
        print(len(lst))
... 100000000
```

```
[3]: print(lst[99])
... 99
```

```
[4]: print(lst[9])
... 9
```

✓ Interactive Computing

✓ Reusable Scripts

Getting started with Jupyter Notebook

Install Jupyter Notebook

Shell

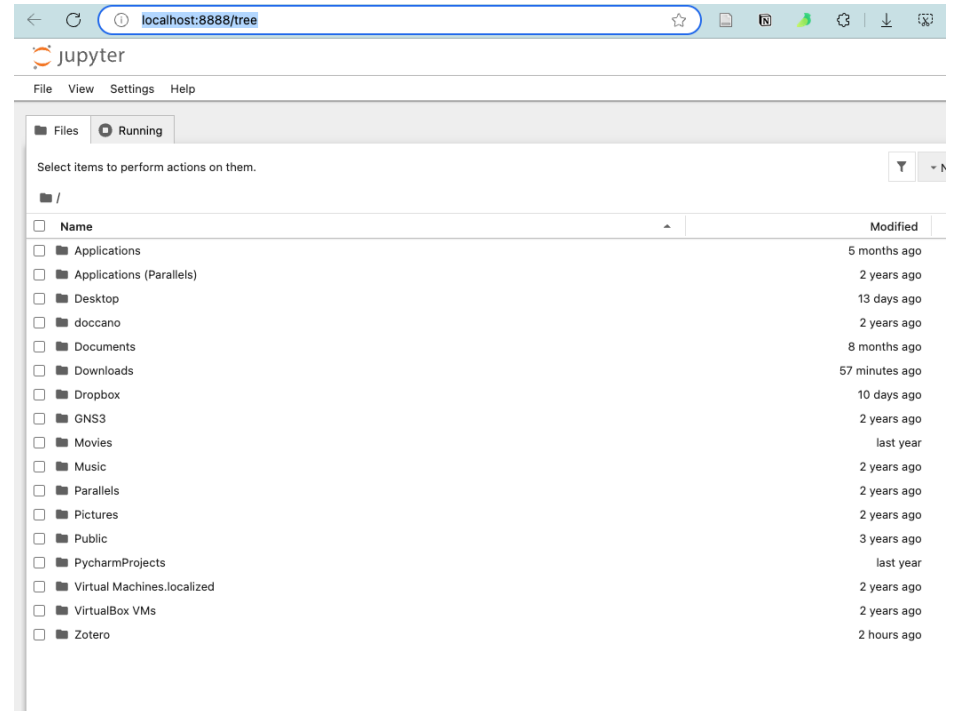
```
$ pip install jupyter
```

Run Jupyter Notebook

```
jupyter-notebook
[I 2025-08-26 16:39:25.790 ServerApp] jupyter_lsp | extension was successfully linked.
[I 2025-08-26 16:39:25.791 ServerApp] jupyter_server_terminals | extension was successfully linked.
[I 2025-08-26 16:39:25.793 ServerApp] jupyterlab | extension was successfully linked.
[I 2025-08-26 16:39:25.794 ServerApp] notebook | extension was successfully linked.
[I 2025-08-26 16:39:25.943 ServerApp] notebook_shim | extension was successfully linked.
[I 2025-08-26 16:39:25.963 ServerApp] notebook_shim | extension was successfully loaded.
[I 2025-08-26 16:39:25.964 ServerApp] jupyter_lsp | extension was successfully loaded.
[I 2025-08-26 16:39:25.965 ServerApp] jupyter_server_terminals | extension was successfully loaded.
[I 2025-08-26 16:39:25.967 LabApp] JupyterLab extension loaded from /Library/Frameworks/Python.framework/Versions/3.13/lib/python3.13/site-packages/jupyterlab
[I 2025-08-26 16:39:25.967 LabApp] JupyterLab application directory is /Library/Frameworks/Python.framework/Versions/3.13/share/jupyter/lab
[I 2025-08-26 16:39:25.967 LabApp] Extension Manager is 'pypl'.
[I 2025-08-26 16:39:25.996 ServerApp] jupyterlab | extension was successfully loaded.
[I 2025-08-26 16:39:25.998 ServerApp] notebook | extension was successfully loaded.
[I 2025-08-26 16:39:25.999 ServerApp] Serving notebooks from local directory: /Users/chenhuangxun
[I 2025-08-26 16:39:25.999 ServerApp] Jupyter Server 2.17.0 is running at:
[I 2025-08-26 16:39:25.999 ServerApp] http://localhost:8888/tree?token=cba55be6152351d9135fe716f3e65856371d91ab2ce3a285
[I 2025-08-26 16:39:25.999 ServerApp] http://127.0.0.1:8888/tree?token=cba55be6152351d9135fe716f3e65856371d91ab2ce3a285
[I 2025-08-26 16:39:25.999 ServerApp] Use Control-C to stop this server and shut down all kernels (twice to skip confirmation).
[C 2025-08-26 16:39:26.003 ServerApp]

To access the server, open this file in a browser:
file:///Users/chenhuangxun/Library/Jupyter/runtime/jpserver-76649-open.html
Or copy and paste one of these URLs:
http://localhost:8888/tree?token=cba55be6152351d9135fe716f3e65856371d91ab2ce3a285
http://127.0.0.1:8888/tree?token=cba55be6152351d9135fe716f3e65856371d91ab2ce3a285
[I 2025-08-26 16:39:26.012 ServerApp] Skipped non-installed server(s): bash-language-server, dockerfile-language-server-nodejs, javascript-typescript-langserver, jedi-language-server, julia-language-server, pyright, python-language-server, python-lsp-server, r-languageserver, sql-language-server, texlab, typescript-language-server, unified-language-server, vscode-css-languageserver-bin, vscode-html-languageserver-bin, vscode-json-languageserver-bin, yaml-language-server
```

Open Jupyter in a web-browser

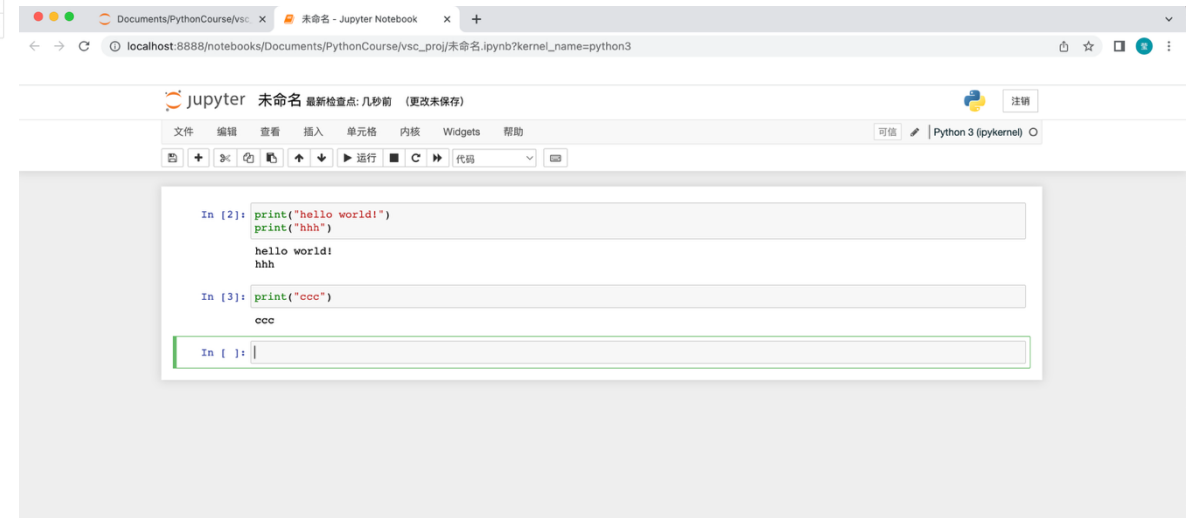
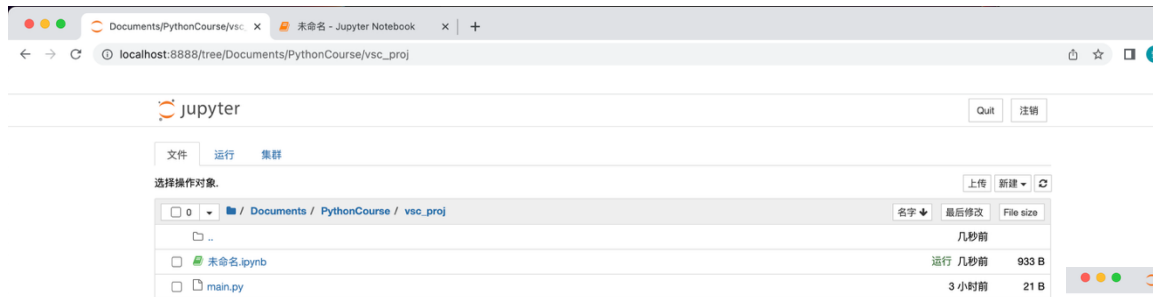


Getting started with Jupyter Notebook

Create an ipynb file and try

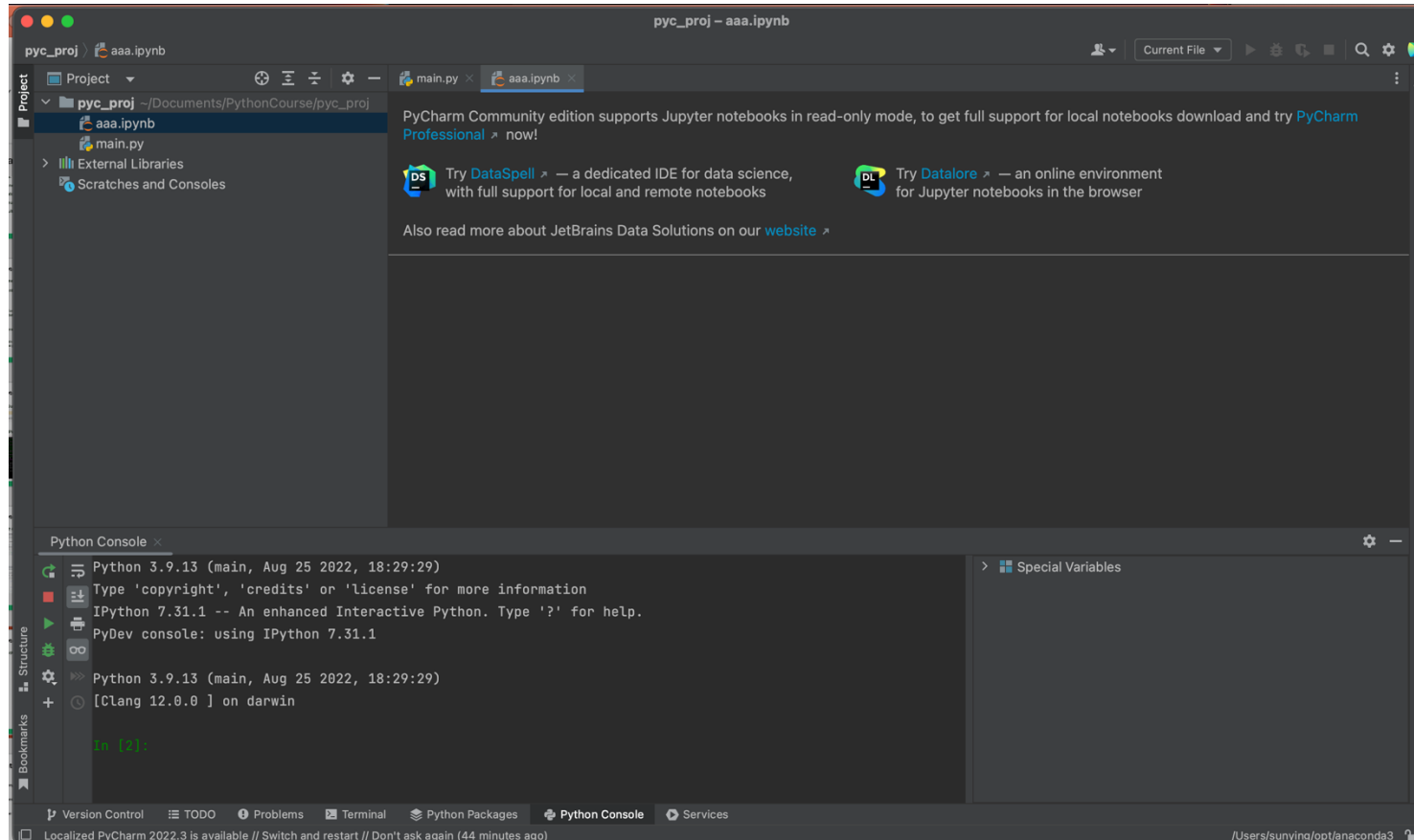
Write and run code in individual cells

View the results instantly within the browser



Jupyter Notebook (PyCharm)

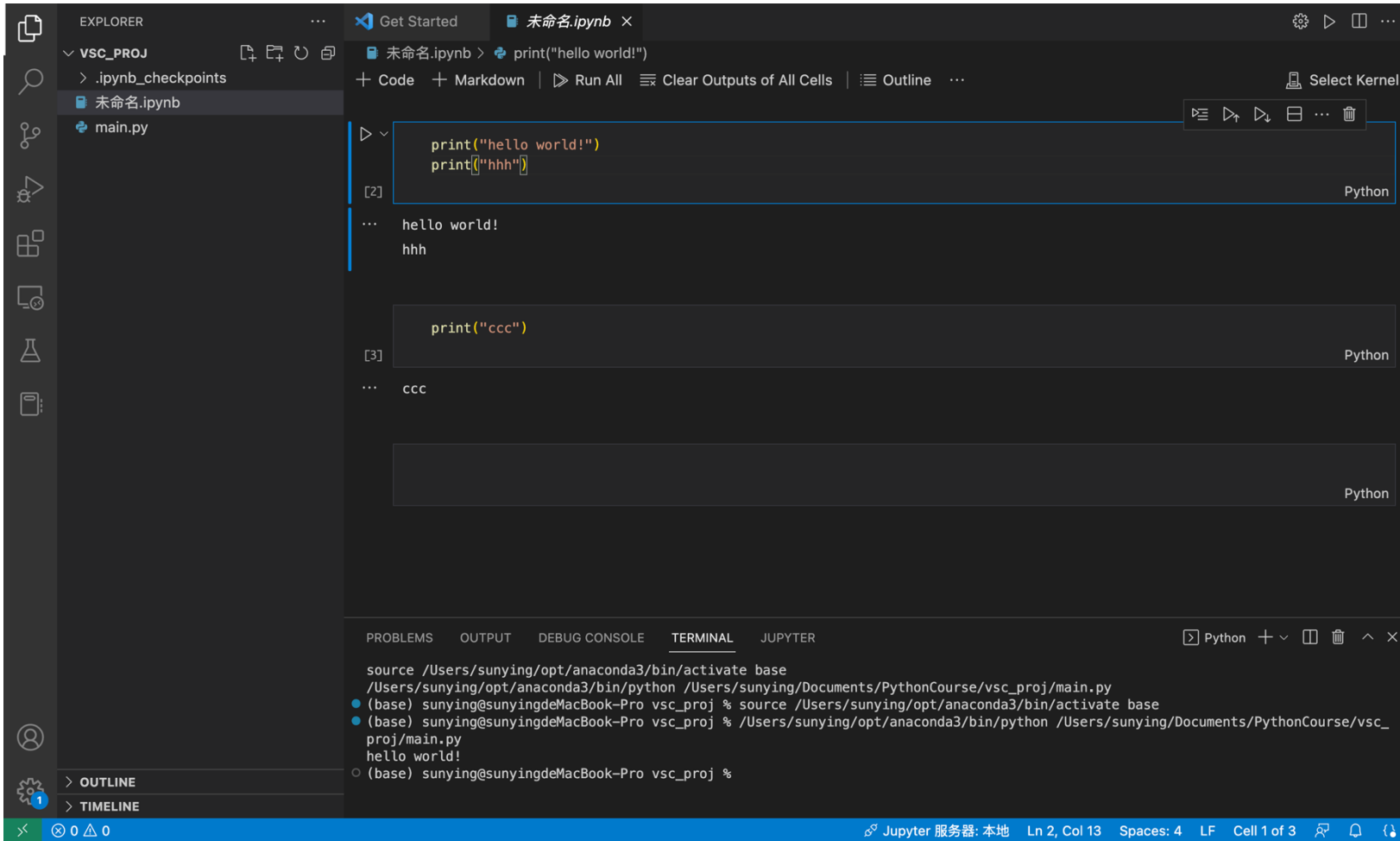
Just create the ipynb file



only for professional

Jupyter Notebook (VScode)

Just create the ipynb file



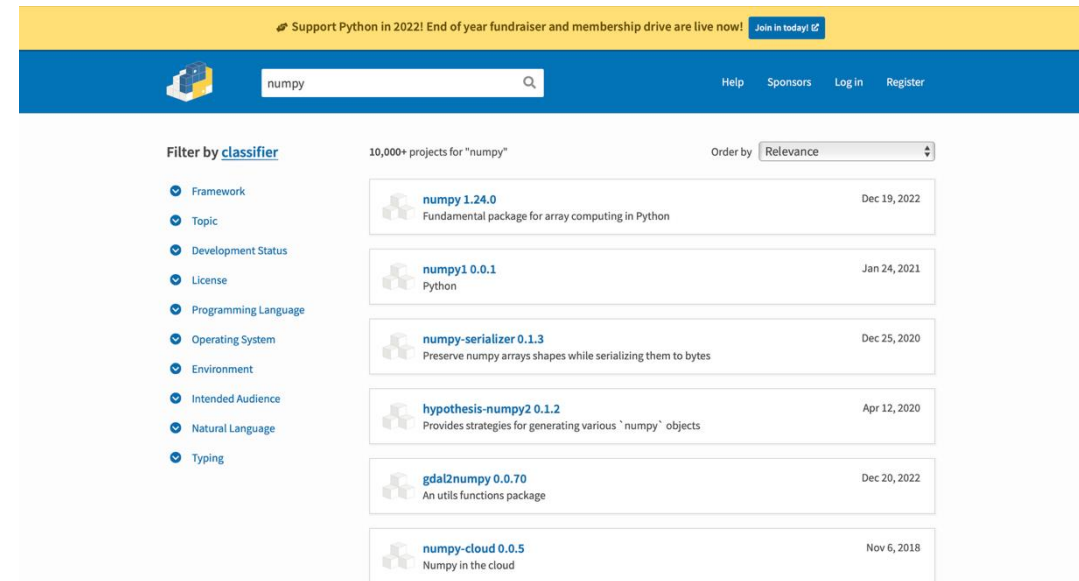
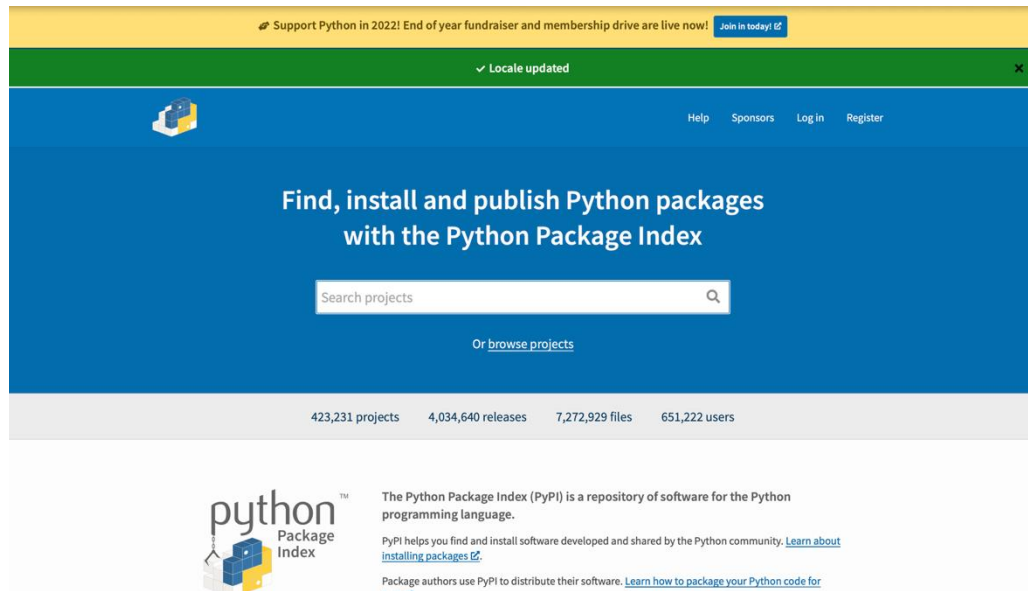
The screenshot displays the VS Code interface with a Jupyter Notebook open. The Explorer sidebar on the left shows a project named 'VSC_PROJ' containing a file 'main.py' and a subdirectory '.ipynb_checkpoints'. The Jupyter Notebook editor shows two cells. The first cell, labeled '[2]', contains the code `print("hello world!")` and `print(["hhh"])`, and its output is `hello world!` and `hhh`. The second cell, labeled '[3]', contains the code `print("ccc")`, and its output is `ccc`. The bottom panel shows the terminal with the following commands and output:

```
source /Users/sunying/opt/anaconda3/bin/activate base
/Users/sunying/opt/anaconda3/bin/python /Users/sunying/Documents/PythonCourse/vsc_proj/main.py
(base) sunying@sunyingdeMacBook-Pro vsc_proj % source /Users/sunying/opt/anaconda3/bin/activate base
(base) sunying@sunyingdeMacBook-Pro vsc_proj % /Users/sunying/opt/anaconda3/bin/python /Users/sunying/Documents/PythonCourse/vsc_proj/main.py
hello world!
```

The status bar at the bottom indicates the current position is Line 2, Column 13, with 4 spaces, in a Python file, Cell 1 of 3.

PyPI - The Python Package Index

The Python Package Index (PyPI) is the official third-party software repository for the Python programming language. It serves as **a central hub where Python developers can publish and share their packages and libraries** with the wider community.



<https://pypi.org>

PyPI - The Python Package Index

Install and use packages in PyPI: pip (installed with Python)

Install: run `pip install <package-name>`

```

pip3 --version
pip 25.2 from /Library/Frameworks/Python.framework/Versions/3.13/lib/python3.13/site-packages/pip (python 3.13)

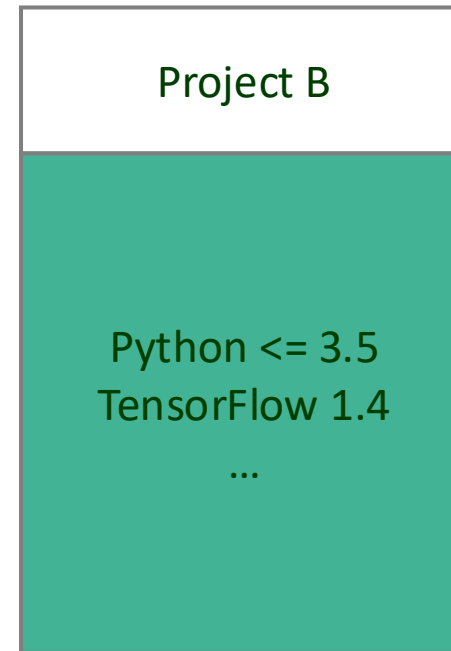
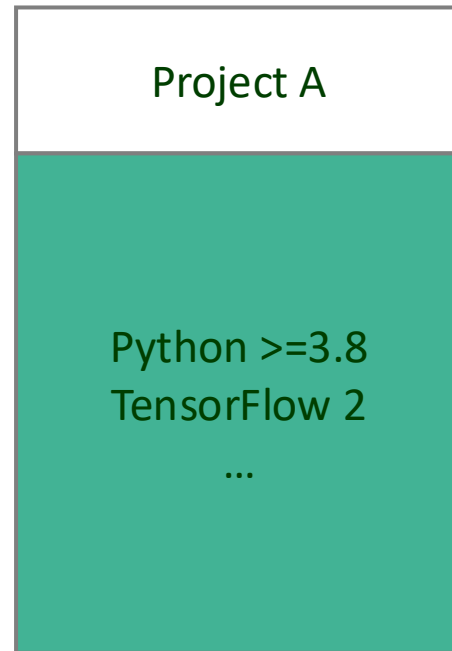
python3
Python 3.13.7 (v3.13.7:bcee1c32211, Aug 14 2025, 19:10:51) [Clang 16.0.0 (clang-1600.0.26.6)] on darwin
Type "help", "copyright", "credits" or "license" for more information.
>>> import numpy
Traceback (most recent call last):
  File "<python-input-0>", line 1, in <module>
    import numpy
ModuleNotFoundError: No module named 'numpy'
>>> exit()

pip3 install numpy
Looking in indexes: https://mirrors.aliyun.com/pypi/simple/
Collecting numpy
  Downloading https://mirrors.aliyun.com/pypi/packages/78/45/d4698c182895af189c463fc91d70805d455a227261d950e4e0f1310c2550/numpy-2.3.2-cp313-cp313-macosx_14_0_arm64.whl (5.1 MB)
    5.1/5.1 MB 6.5 MB/s 0:00:00
Installing collected packages: numpy
Successfully installed numpy-2.3.2

python3
Python 3.13.7 (v3.13.7:bcee1c32211, Aug 14 2025, 19:10:51) [Clang 16.0.0 (clang-1600.0.26.6)] on darwin
Type "help", "copyright", "credits" or "license" for more information.
>>> import numpy
>>>
```

Manage multiple Python environments in isolation, efficiently and without conflicts

- <https://docs.conda.io/projects/conda/en/latest/user-guide/install/index.html>
- Anaconda: <https://www.anaconda.com/>



Conflict?

- Create an environment
- Activate the environment
- Install packages
- Remove packages
- Remove an environment

```
# create an environment
```

```
conda create -n env1 python=3.6
```

```
# switch the environment
```

```
conda activate -n env1
```

```
# install packages
```

```
conda install seaborn
```

```
# remove an environment
```

```
conda remove -n env1 --all
```



GitHub Copilot Setup

- Copilot provides AI-powered code suggestions and chat assistance inside coding environments.
- It can help with code completion, code explanation, debugging, test generation, and documentation.
- Copilot helps you write code, but you are responsible for understanding, testing, and validating the code.

Github Copilot General Setup Workflow

1. Create or sign in to a GitHub account.
2. Apply for GitHub Education / Copilot Student access.
3. Install an IDE or notebook environment. (We have already done)
4. Install the GitHub Copilot extension/plugin.
5. Sign in with GitHub in the IDE.
6. Test inline suggestions and Copilot Chat.

Github Copilot Registration

- Go to GitHub Education Student benefits. <https://github.com/education/students>
- Sign in with your GitHub account.
- Verify your student status using your university email or required documents.
- After approval*, enable GitHub Copilot Student access.
- Use the same GitHub account to sign in from VS Code or PyCharm.

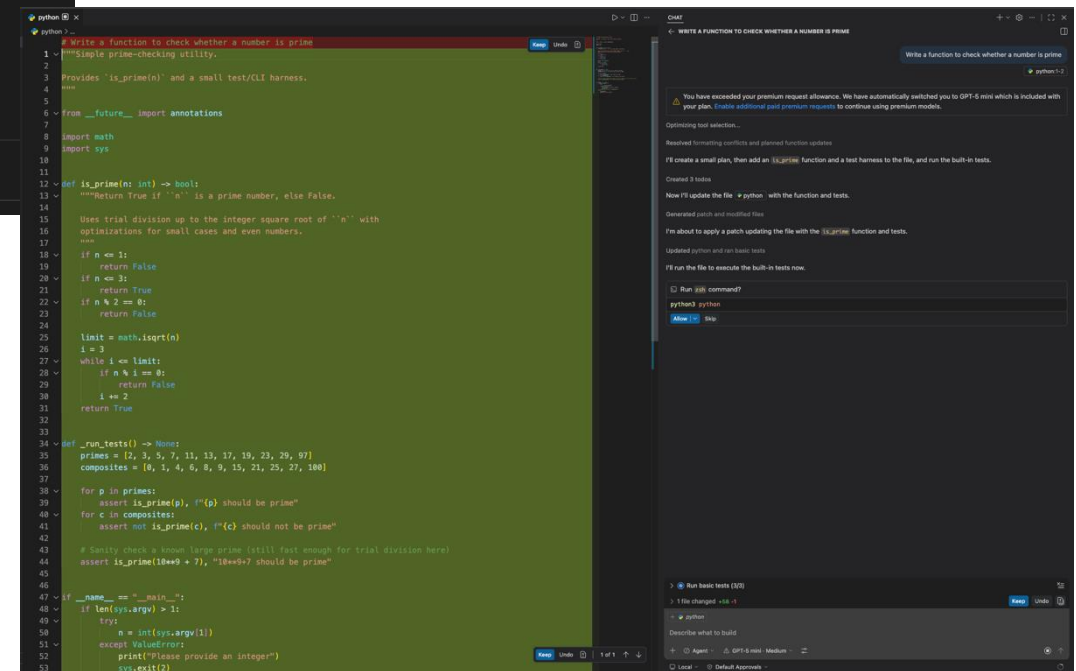
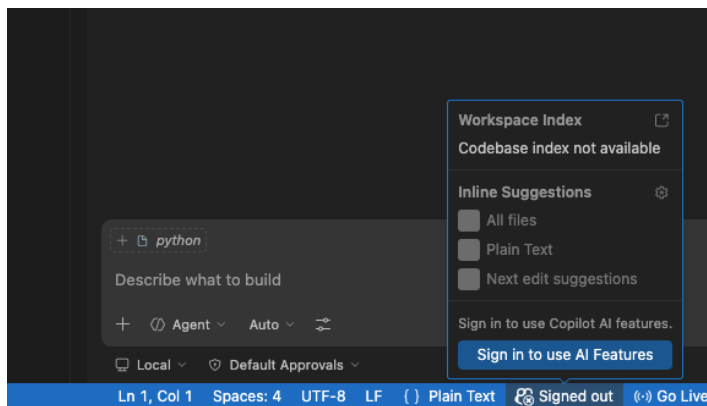
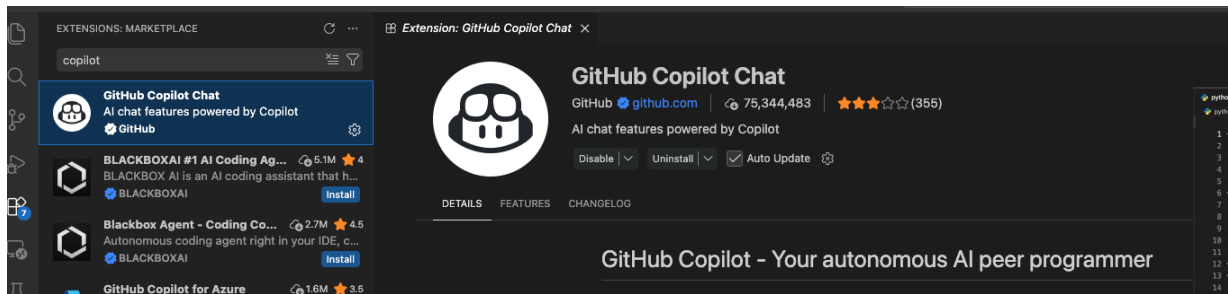
* Approval may take time.

* **There is also free plan to get started.**

A Chinese tutorial: https://blog.csdn.net/PGJ_168/article/details/150267222

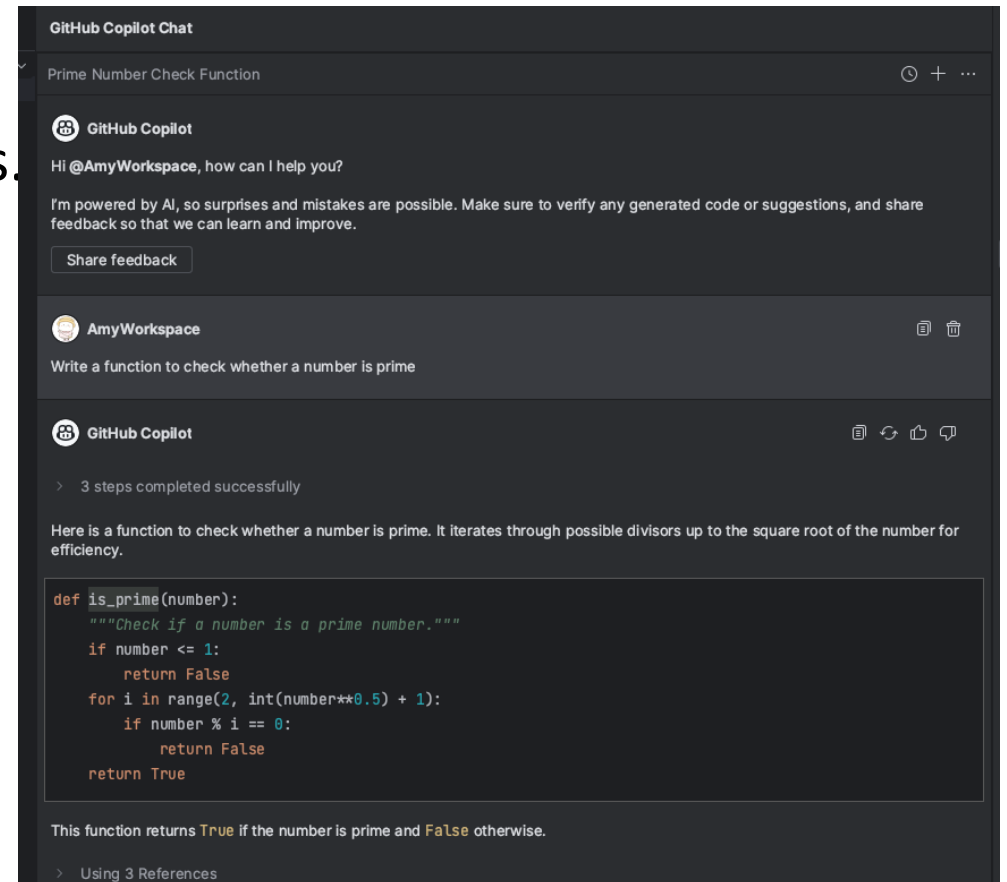
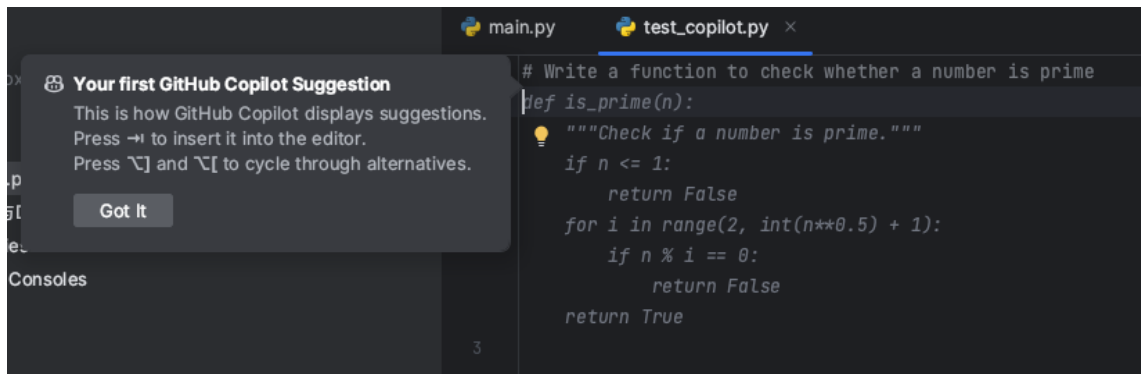
Github Copilot + VS Code

- Setup VS code first
- Install the GitHub Copilot extension.
- Sign in with your GitHub account.
- Open a .py file and test Copilot suggestions.



Github Copilot + PyCharm

- Setup PyCharm first
- Go to Settings / Preferences → Plugins, and install the Copilot plugin.
- Restart PyCharm if required.
- Sign in with your GitHub account.
- Open a Python file and test copilot suggestions.



- NOT support browser-only Jupyter Notebook
- Recommended method: use Jupyter Notebook inside VS Code/PyCharm
- Open or create a .ipynb file in IDE.
- Sign in to GitHub.
- Try Copilot suggestions inside notebook code cells.

Allowed

- Use Copilot to understand examples.
- Use Copilot to get coding hints.
- Use Copilot to debug error messages.
- Use Copilot to generate possible test cases.
- Use Copilot to improve readability after you understand the code.

Not Allowed

- Submit code you cannot explain.
- Submit AI-generated code without testing.
- Use Copilot to bypass independent work.
- Copy complete solutions without understanding.

You may use AI tools as assistants, but **you are fully responsible for your submitted code.**



Simple LLM API Call demo

Why Call an LLM API?

API call is the basic building block of AI applications and agents. LLM API helps our code use AI capabilities.

Relationship with Copilot:

Direct LLM API call: you write the request yourself.

Copilot: the IDE/plugin writes and manages the request for you.

What Is an LLM API Call?

Your Python program sends a request to the LLM server.

The request includes:

- API URL, API key, model name
- system prompt, user prompt
- some parameters, e.g., temperature

The server returns a response, usually in JSON format.

A Minimal Example

```
import requests
import json

url = "https://aigc-api.hkust-gz.edu.cn/v1/chat/completions"

headers = {
    "Content-Type": "application/json",
    "Authorization": api_key
}

data = {
    "model": "DeepSeek-V4-Pro",
    "messages": [
        {"role": "system", "content": "You are a helpful coding assistant."},
        {"role": "user", "content": "Write a Python program to print all prime numbers between 0 and 100."}
    ],
    "temperature": 0.7
}

response = requests.post(url, headers=headers, data=json.dumps(data))
```

The API key should not be hard-coded in the source code.

A Minimal Example: Reading API Key from a File

```
def load_api_key(key_file):  
    with open(key_file, "r", encoding="utf-8") as f:  
        return f.read().strip()  
  
api_key = load_api_key("api_key.txt")
```

- Store the key in api_key.txt.
- Do not share the key.
- Do not upload the key to GitHub.
- Add api_key.txt to .gitignore.

A Minimal Example: Parse and Use the Response

```
response_json = response.json()
message = response_json["choices"][0]["message"]["content"]
print(message)
```

Calling an LLM API is not only about getting text.

It is the first step toward building AI-powered Python applications.

Revisiting Course Overview and Agenda

Week	Date	Title	
1	2026-06-15	Introduction	
2	2026-06-17	Advanced Python Fundamentals-I	Part 1: Advanced Python Fundamentals
3	2026-06-22	Advanced Python Fundamentals-II	
4	2026-06-24	Advanced Python Fundamentals-III	
5	2026-06-29	Machine Learning I – workflow, supervised v.s. unsupervised	Part 2: Python for ML/DL/LLM
6	2026-07-01	Machine Learning II - evaluation, regularization, ensembles, ...	
7	2026-07-06	Deep Learning I: PyTorch (tensors, autograd, NN)	
8	2026-07-08	Deep Learning-II: architectures, optimization, training, debugging	
9	2026-07-13	Deep Learning-III: LLMs, transformers, pretrain, fine-tuning, inference	
10	2026-07-15	AI-powered Application I: prompting, structured outputs, ...	Part 3: AI-powered and Agentic Applications
11	2026-07-20	AI-powered Applications II: RAG, tool use, agentic workflows, etc, ...	
12	2026-07-22	AI-powered Applications III: AI-assisted Coding	
13	2026-07-27	Final Project Presentation - presentations, reflection, wrap-up	Part 4: Assessment
14	TBD	Final Exam	

Do not outsource your thinking too early.

1. Think first

Understand the problem, design your own solution, and write your first version of the code.

2. Then compare with AI

Ask AI to solve the same problem and compare its solution with yours.

3. Ask why

Do not only ask AI for the answer. Ask:

- Why does this solution work?
- What is different from my solution?
- Which implementation is more readable or efficient?
- Are there any hidden bugs or edge cases? Etc.

4. Use AI as a learning partner

AI can explain, debug, suggest alternatives, and generate tests — but you are responsible for understanding and validating the code.

Your programming mental muscle needs training. Use AI to strengthen it, not to replace it.

Revisiting today's content

- Course Information
- Introduction to Python
- Python and IDE Setup
- GitHub Copilot Setup
- Simple LLM API Call demo

If you decide to continue with this course, please make sure that, before the next lecture, you are familiar with Python environment, have chosen your preferred IDE, set up Copilot, and mastered the basic LLM API call.

Python Programming with Artificial Intelligence

Thank you for listening!

